Freestanding Library: Easy [utilities], [ranges], and [iterators]

Document number: P1642R7
Date: 2021-09-26
Reply-to: Ben Craig <ben dot craig at gmail dot com>
Audience: Library Evolution Working Group

Change history

R7

- Fixed wording to keep `<atomic>` freestanding
- Mentioned reflector discussion around addition of `std::unreachable`

R6

- Adding the accidentally omitted `addressof`. Earlier versions of the paper included it, and it was lost due to shifts in this paper's wording, and the relocation of `addressof` out of [specialized.algorithms]

R5

- Rebasing to N4868
- Adding `_Exit`, as `quick_exit` is specified in terms of `_Exit`
- Explicitly listing out the algorithms from [specialized.algorithms] instead of referring to them by the stable name tag

R4

- Moved feature test macros to P2198
- Pulled in wording approach from P1641. Abandoning the P1641 paper number
- Adding `make_obj_using_allocator` and `uninitialized_construct_using_allocator`. These do not allocate memory using an allocator, they construct a `T` while passing along an allocator when possible
- Including additional rationale about the omission of allocating functions
- Added editorial alternatives
- Added implementation and deployment experience

R3

- Rebased to N4861
- Putting wording back (in the form of instructions to editor)
- Per-header feature test macro, similar to `constexpr` macros

R2

- Rebased to N4842
- No longer including `istream_view`

R1

- Adding section summarizing what is getting added
- Adding all of [ranges] and most of [iterators]
- Adding `uses_allocator_construction_args`
- Adding feature test macro
- Now discussing split overload sets
- Dropping wording for now

R0

- Branching from P0829R4. This "omnibus" paper is still the direction I am aiming for. However, it is too difficult to review. It needs to change with almost every meeting. Therefore, it is getting split up into smaller, more manageable chunks
- Limiting paper to the [utilities], [ranges], and [iterators] clauses

Introduction

This paper proposes adding many of the facilities in the [utilities], [ranges], and [iterators] clause to the freestanding subset of C++. The paper will be adding only complete entities, and will not tackle partial classes. For example, classes like `pair` and `tuple` are being added, but trickier classes like `optional`, `variant`, and `bitset` will come in another paper.

The `<memory>` header depends on facilities in `<ranges>` and `<iterator>`, so those headers (and clauses) are addressed as well.

This paper also updates the editorial techniques for declaring facilities as freestanding. This update will make it easier to mark headers as partially freestanding. This wording change will also make it easier to mark in-flight proposals as freestanding, without causing a major blocking point in the standardization process.

The paper also includes a drive-by fix to add `_Exit` to freestanding. `quick_exit` is already part of C++20's freestanding library, and `quick_exit` is specified to call `_Exit` [support.start.term#13]. This has been a bug since C++11 (introduced with LWG1264 / UK-172).

Changes to the design of feature test macros are discussed in P2198. This paper maintains the status quo for feature test macros on freestanding.

Motivation and Design

Many existing facilities in the C++ standard library could be used without trouble in freestanding environments. This series of papers will specify the maximal subset of the C++ standard library that does not require an OS or space overhead.

For a more in-depth rationale, see P0829.

This paper adds blanket wording to make it easier to mark things freestanding, and it retrofits the existing freestanding facilities to use the new editorial approach. It also includes a non-normative note to allow implementers to use existing no-exception builds of their libraries as an implementation-defined extensions, with a fair number of qualifications. Here's that note, reproduced so that it can be discussed more effectively:

[*Note:* Throwing a standard library provided exception is not observably different from `terminate()` if the implementation doesn't unwind the stack during exception handling ([except.handle#9]) and the user's program contains no catch blocks. *-end note*]

`<optional>`, `<variant>`, and `<bitset>` are not discussed in this paper, as all have non-essential functions that can throw an exception. `<charconv>` is also not discussed in this paper as it will require us to deal with the thorny issue of overload sets involving floating point and non-floating point types. I plan on addressing all four of these headers in later papers, so that the topics in question can be discussed in relative isolation.

Splitting overload sets

Modifying an overload set needs to be treated with great care. Ideally, libraries built in freestanding environments will have the same semantics (including selected overloads) when built in a hosted environment. I doubt that goal is fully attainable, as sufficiently clever programmers will be able to detect the difference and modify behavior accordingly.

My approach will be to avoid splitting overload sets that could cause accidental freestanding / hosted differences. (In future papers, I may need to lean on `= delete` techniques to avoid silent behavior changes, but this paper hasn't needed that approach.)

`swap` has `shared_ptr`, `weak_ptr`, and `unique_ptr` overloads, but this paper marks only the `unique_ptr` overload as freestanding. `<memory>` has many algorithms with `ExecutionPolicy` overloads. This paper does not mark the `ExectuionPolicy` overloads as freestanding. I was unable to come up with any compelling "accidental" way to select one `swap` or algorithm overload in freestanding, but a different one in hosted.

Also note that the `swap` overload set visible to a given translation unit is already indeterminate. A user may include `<memory>` which guarantees the smart pointer overloads, but an implementation could expose any (or none!) of the other `swap` overloads from other STL headers.

Memory allocation and allocators

P2013 makes the default allocating `::operator news` optional in freestanding. This would make `std::allocator::allocate` ill-formed on implementations without an `::operator new` definition.

This paper (and most of the freestanding papers) will attempt to avoid adding facilities to freestanding that throw exceptions.

The Cpp17Allocator requirements permit only one and a half ways of signaling failures. The first way to signal a failure is by throwing an exception; this is the only way to signal a failure that is explicitly described in [allocator.requirements]. The "half" of a way to signal a failure from an allocator is to `terminate`. This can happen if the implementation has exceeded its (potentially tiny) implementation limits. Note that returning `nullptr` is not an option here, as making `nullptr` a standard permitted error handling strategy for `std::allocator::allocate` would be a massively breaking change.

Future papers will attempt to add `constexpr` facilities to freestanding, so long as they are evaluated at compile time only. The intent is for these future papers to allow a compile-time `std::vector` and `std::allocator`, but without requiring a runtime `std::vector` or `std::allocator`.

This paper shouldn't make any of those efforts more difficult to address in a distinct paper. WG21 should not wait for those problems to be solved before adding the facilities in this paper to freestanding.

Justification for omissions

The following functions and classes rely on dynamic memory allocation and exceptions:

- `make_unique`
- `make_unique_for_overwrite`
- `shared_ptr`
- `weak_ptr`
- `function`
- `boyer_moore_searcher`
- `boyer_moore_horspool_searcher`

The following classes rely on iostreams facilities. iostreams facilities use dynamic memory allocations and rely on the operating system.

- `istream_iterator`
- `ostream_iterator`
- `istreambuf_iterator`
- `ostreambuf_iterator`
- `basic_istream_view`
- `istream_view`

The ExecutionPolicy overloads of algorithms are of minimal utility on systems that do not support C++ threads.

Feature test macros

Feature test macros are non-trivial for this new design space, and are now discussed in paper P2198. For now, the present paper maintains the status quo with regards to the contents of `<version>`.

Dependency on `optional`-like thing in ranges

[range.semi.wrap] describes an exposition-only type with behavior specified in terms of `std::optional`. This exposition-only type is used to specify `single_view`, `filter_view`, `transform_view`, `take_while`, and `drop_while_view`. `std::optional` is not added in this paper.

I claim that this is not a problem. First, the type `std::optional` is not required to be used here. Second, [compliance] paragraph 2 permits implementations to provide extra headers, and an implementation could provide `std::optional` if it so desired. Third, I plan on getting `std::optional` added to freestanding in a future paper that tackles the issue of omitting potentially throwing members, like `optional::value`.

Implementation and field experience

NI has been using a modified form of STLPort in the Microsoft Windows, Linux, and Apple OSX kernel for more than 10 years. That only covers C++03 facilities though.

Paul Bendixen has implemented P0829 in a libstdc++ fork. P0829 is (almost) a superset of the facilities in this paper. This implementation is available on Conan.

I have run a 2018 era of libc++'s test suite against the Microsoft Visual Studio 15.8 (released Aug 14, 2018) STL implementation, run in the Microsoft Windows kernel. Running in that environment allowed me to audit whether exceptions, RTTI, memory allocation, or floating point was used in appropriately. This largely tested the C++14 parts of the STL. The C++17 and C++20 parts were either missing implementations, or tests.

Post-LEWG merge conflicts

This paper was last discussed in LEWG in December 2020, and forwarded to LWG in March 2021. The paper touches a lot of headers, and the standard is a moving target.

P0627: Function to mark unreachable code is likely to make it into the standard at a similar time as this paper. P0627 would add `std::unreachable` to `<utility>`. Since this paper adds the entirety of `<utility>` to freestanding, that constitutes a design change. A thread was started on the reflector asking if there was any opposition to adding `std::unreachable` to freestanding. There were five votes in favor of putting `std::unreachable` in freestanding, and no opposition (the author of this paper did not vote).

Editorial alternatives

This paper uses an opt-in editorial syntax like the following:

```
#define NULL see below // freestanding
```

This gives normative meaning to a comment in the synopsis. There is precedent for this, particularly for `// exposition only` comments.

An alternative used by the POSIX spec would be to use an extra-lingual annotation to mark entities as freestanding or not. Here, I'll use an italicized `[FS]` to indicate freestanding membership.

```
[FS] #define NULL see below
```

The choices of square brackets and the "FS" tag are arbitrary . Other separators or tags could be used instead. This would be a substantially new editorial technique for the C++ standard.

We could continue with the current approach, and use prose to indicate freestanding membership, similar to what we do with `abort` and friends in [compliance]. This would be a substantial amount of specification work for some headers. It also hides the choice of freestanding, such that future paper authors are even less likely to keep it in mind.

When picking an editorial alternative, we should keep in mind likely future needs. P0829 makes use of `//freestanding, partial` and `//freestanding, omit` annotations. P1641R3 makes use of `//freestanding only` annotations. I can also see uses for a `//freestanding delete` annotation, for maintaining overload sets while removing functionality. These could be represented in the POSIX-like annotation as follows: `[FS partial]`, `[FS omit]`, `[FS only]`, and `[FS delete]`.

Status Quo

Please see [compliance] to see what is freestanding in C++20. Note that `NULL` and `size_t` are declared in both `<cstdlib>` and `<cstddef>`. The wording in [compliance] does not make it clear that `<cstdlib>` should continue to provide those declarations in a freestanding implementation.

Wording

The following wording is relative to N4868.

Change in [conventions]

Add a new subclause [freestanding.membership], under [conventions] and after [objects.within.classes]:

?.?.?.? **Freestanding membership** **[freestanding.membership]**

1 Unless otherwise specified, the requirements on declarations and macro definitions in freestanding implementations shall be the same as the corresponding requirements in a hosted implementation.
[*Note:* Throwing a standard library provided exception is not observably different from `terminate()` if the implementation doesn't unwind the stack during exception handling ([except.handle#9]) and the user's program contains no catch blocks. -*end note*]
In the associated header synopsis for such declarations and macro definitions, the items are followed with a comment that includes *freestanding*.
[*Example:*

```
#define NULL see below // freestanding
```

-*end example*]

2 A header all of whose declarations and macro definitions would be commented as specified above shall instead have a single such comment at the beginning of its synopsis.
[*Example:*

```
// freestanding
namespace std {
```

-*end example*]

3 A *freestanding class template* is a class template that is implemented (partially or fully) in freestanding implementations. Deduction guides for freestanding class templates shall be implemented in freestanding implementations.

4 A *freestanding declaration* is a non-namespace declaration that is implemented (partially or fully) in a freestanding implementation. The containing namespace of each freestanding declaration shall be provided in a freestanding implementation.

Wording involving comment placement is inspired by [expos.only.func]#1. The "shalls" are all requirements on the implementation, and are similar to [global.functions]#3.

Changes in [compliance]

Add new rows to Table 24:

Table 24: C++ headers for freestanding implementations [tab:headers.cpp.fs]

	Subclause	Header(s)
[…]	[…]	[…]
?.? [utility]	**Utility components**	**`<utility>`**
?.? [tuple]	**Tuples**	**`<tuple>`**
?.? [memory]	**Memory**	**`<memory>`**
?.? [function.objects]	**Function objects**	**`<functional>`**
?.? [ratio]	**Compile-time rational arithmetic**	**`<ratio>`**
?.? [iterators]	**Iterators library**	**`<iterator>`**
?.? [ranges]	**Ranges library**	**`<ranges>`**
[…]	[…]	[…]

Change in [compliance] paragraph 3:

~~The supplied version of the header <cstdlib> shall declare at least the functions abort, atexit, at_quick_exit, exit, and quick_exit ([support.start.term]). The supplied version of the header <atomic> shall meet the same requirements as for a hosted implementation except that support for always lock-free integral atomic types ([atomics.lockfree]) is implementation-defined, and whether or not the type aliases atomic_signed_lock_free and atomic_unsigned_lock_free are defined ([atomics.alias]) is implementation-defined. The other headers listed in this table shall meet the same requirements as for a hosted implementation.~~ **The headers listed in Table 24 shall meet the requirements for a freestanding implementation, as specified in the respective header synopsis.**

Full headers

Instructions to the editor:
Please insert a *// freestanding* comment at the beginning of the following synopses. These headers are entirely freestanding.

- [cstddef.syn]
- [version.syn]
- [limits.syn]
- [climits.syn]
- [cfloat.syn]
- [cstdint.syn]
- [new.syn]
- [typeinfo.syn]
- [source.location.syn]
- [exception.syn]
- [initializer.list.syn]
- [compare.syn]
- [coroutine.syn]
- [cstdarg.syn]
- [concepts.syn]
- [utility.syn]
- [tuple.syn]
- [meta.type.synop]
- [ratio.syn]
- [bit.syn]

Change in [cstdlib.syn]

Instructions to the editor:
Please append a *// freestanding* comment to the following entities:

- `size_t`
- `NULL`
- `abort`
- all overloads of `atexit`
- all overloads of `at_quick_exit`
- `exit`
- `_Exit`
- `quick_exit`

Change in [memory.syn]

Instructions to the editor:
Please append a *// freestanding* comment to the following entities:

- `pointer_traits`
- `to_address`
- `align`
- `assume_aligned`
- `allocator_arg_t`
- `allocator_arg`
- `uses_allocator`
- `uses_allocator_v`
- `uses_allocator_construction_args`
- `make_obj_using_allocator`
- `uninitialized_construct_using_allocator`
- `allocator_traits`
- `addressof`
- `default_delete`
- `unique_ptr`
- `unique_ptr` overload of `swap`
- relational operators (including three-way / spaceship) involving `unique_ptr`
- `hash`
- `unique_ptr` specialization of `hash`
- `atomic`

Please append a *// freestanding* comment to all overloads of the following function templates, except for the ExecutionPolicy overloads. This includes the function templates in the `ranges` namespace.

- `uninitialized_default_construct`
- `uninitialized_default_construct_n`
- `uninitialized_value_construct`
- `uninitialized_value_construct_n`
- `uninitialized_copy_result`
- `uninitialized_copy`
- `uninitialized_copy_n_result`
- `uninitialized_copy_n`
- `uninitialized_move_result`
- `uninitialized_move`
- `uninitialized_move_n_result`
- `uninitialized_move_n`
- `uninitialized_fill`
- `uninitialized_fill_n`
- `construct_at`
- `destroy_at`
- `destroy`
- `destroy_n`

Note: the following portions of `<memory>` are **omitted**. No change to the working draft should be made for these entities:

- [util.dynamic.safety], pointer safety
- `allocator` and associated comparisons
- `ExecutionPolicy` overloads in [specialized.algorithms]

- `make_unique`
- `make_unique_for_overwrite`
- All `operator<<` overloads
- `bad_weak_ptr`
- `shared_ptr`
- `make_shared`
- `allocate_shared`
- `make_shared_for_overwrite`
- `allocate_shared_for_overwrite`
- relational operators (including three-way / spaceship) involving `shared_ptr`
- `shared_ptr` overload of `swap`
- `static_pointer_cast`
- `dynamic_pointer_cast`
- `const_pointer_cast`
- `reinterpret_pointer_cast`
- `get_deleter`
- `weak_ptr`
- `weak_ptr` overload of `swap`
- `owner_less`
- `enable_shared_from_this`
- `shared_ptr` specialization of `hash`
- `shared_ptr` specialization of `atomic`
- `weak_ptr` specialization of `atomic`

Change in [functional.syn]

Instructions to the editor:
Please append a *// freestanding* comment to every entity in `<functional>` **except** for the following entities:

- `bad_function_call`
- `function`
- `function` overloads of `swap`
- `function` overloads of `operator==`
- `boyer_moore_searcher`
- `boyer_moore_horspool_searcher`

Change in [iterator.synopsis]

Instructions to the editor:
Please append a *// freestanding* comment to every entity in `<iterator>` **except** for the following entities:

- `istream_iterator` and associated comparison operators
- `ostream_iterator`
- `istreambuf_iterator` and associated comparison operators
- `ostreambuf_iterator`

Change in [ranges.syn]

Instructions to the editor:
Please append a *// freestanding* comment to every entity in `<ranges>` **except** for the following entities:

- `basic_istream_view`
- `istream_view`

Change in [atomics.syn]

Instructions to the editor:
Please append a *// freestanding* comment to every entity in `<atomic>` **except** for the following entities:

- `atomic_signed_lock_free`
- `atomic_unsigned_lock_free`

Change in [atomics.lockfree], paragraph 2

> **On hosted implementations ([compliance]), a**~~A~~t least one signed integral specialization of the `atomic` template, along with the specialization for the corresponding unsigned type ([basic.fundamental]), is always lock-free.
> ~~[*Note*~~
> ~~: This requirement is optional in freestanding implementations ([compliance]). — *end note*~~
> ~~]~~

Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing P0829.

Thanks to Odin Holmes for providing feedback and helping publicize P0829.

Thanks to Paul Bendixen for providing feedback while prototyping P0829.

Thanks to Walter Brown for providing wording feedback.

Similar work was done in the C++11 timeframe by Lawrence Crowl and Alberto Ganesh Barbati in N3256.